

PhishShield AI An Intelligent MultiLayer Phishing URL Detection System

Rohith V Kulkarni
Akshay Kashyap M
Arya H

*Dept. of Artificial Intelligence and Data Science
Reva University
BangaloreIndia*

Abstract

Phishing attacks continue to pose a significant threat to cybersecurity with attackers employing increasingly sophisticated techniques to deceive users into revealing sensitive information. This paper presents PhishShield AI an intelligent multilayer phishing URL detection system that leverages ensemble machine learning techniques combined with comprehensive URL feature engineering. The system extracts over 30 features from URLs including lexical structural statistical and securitybased attributes and employs a multimodel evaluation pipeline using Random Forest Gradient Boosting XGBoost and LightGBM classifiers to achieve optimal detection accuracy. The bestperforming model is automatically selected based on F1score and deployed in a realtime detection pipeline integrated with a browser extension and web dashboard. Experimental results demonstrate that the proposed system achieves high detection accuracy with low falsepositive rates providing an effective defense mechanism against phishing threats. The system architecture supports realtime URL scanning user feedback collection and continuous model improvement through an integrated fullstack platform.

Keywords *phishing detection machine learning URL analysis ensemble learning cybersecurity feature engineering web security*

I. INTRODUCTION

Phishing remains one of the most prevalent and dangerous forms of cyber attacks in the digital landscape. According to the AntiPhishing Working Group (APWG) phishing attacks reached an alltime high in recent years with millions of unique phishing URLs detected globally. These attacks exploit human psychology by impersonating trusted entities through deceptive websites leading to credential theft financial fraud and data breaches.

This paper presents PhishShield AI a comprehensive phishing detection system that addresses these limitations through three key contributions (1) a comprehensive feature engineering pipeline that extracts over 30 URLbased features across six distinct categories (2) a multimodel ensemble evaluation framework that automatically selects the bestperforming classifier and (3) a fullstack deployment architecture featuring a browser extension web dashboard and sandbox analysis environment for realtime phishing detection.

The socioeconomic impact of phishing has reached unprecedented levels with cybercriminals leveraging generative AI

to massproduce highly convincing deceptive content at scale. Enterprises face not only direct financial losses but also severe reputational damage regulatory fines for data breaches and the hidden costs of incident response. A successful spearphishing campaign against key employees can completely bypass sophisticated network perimeter defenses providing attackers with valid credentials that grant unfettered access to sensitive internal corporate systems proprietary source code and customer databases. Furthermore the advent of generative AI technologies has significantly lowered the barrier to entry. Attackers can now programmatically generate thousands of evasive domain variations and polymorphic URL structures. This asymmetry requires defensive systems to be equally dynamic leveraging advanced ensemble models and continuous feature engineering to counter automated attack pipelines effectively in realtime.

II. LITERATURE SURVEY

Phishing detection techniques can be broadly categorized into four major approaches listbased methods visual similaritybased methods contentbased analysis and machine learningbased classification. Each approach has its own strengths and limitations and recent research has focused on combining these approaches for improved performance.

Jain and Gupta [1] proposed a machine learningbased phishing detection system using URL features and achieved 95.2% accuracy with Random Forest. However their feature set was limited to basic URL characteristics. Sahingoz et al. [2] introduced an NLPbased approach using word embeddings on URL tokens achieving 97.3% accuracy but with higher computational cost. Rao and Pais [3] demonstrated the effectiveness of combining URL features with page content analysis reporting 96.6% accuracy.

More recently deep learning approaches have shown promising results. Le et al. [4] applied convolutional neural networks on raw URL character sequences and achieved 98.1% accuracy. Bahnsen et al. [5] used LSTM networks for URLbased detection reporting 98.7% accuracy. However these deep learning methods require significant computational resources specialized GPU hardware for inference and exceptionally large training datasets that are often unavailable to independent researchers.

Furthermore the opacity of deep neural networks introduces a critical blackbox problem. In cybersecurity explainability is often as important as raw accuracy. Security Operations Center (SOC) analysts require contextual evidence to understand why a URL was blocked in order to manage false positive escalations effectively. Treebased ensemble methods while slightly less adept at automatic sequence feature extraction than RNNs offer native

feature importance metrics and decision path tracing making them significantly more suitable for enterprise deployment. Recent hybrid studies suggest that intelligently engineered statistical features processed by XGBoost can rival the performance of heavy deep learning pipelines at a fraction of the computational cost.

Wei et al. [6] combined URL lexical features with DNS metadata using XGBoost achieving 97.1% accuracy with improved robustness against adversarial URL manipulation. Tang and Mahmoud [7] conducted a comprehensive survey of MLb phishing solutions highlighting that ensemble methods consistently outperform individual classifiers across diverse datasets.

Despite these advances most existing systems suffer from one or more limitations reliance on single classifiers narrow feature sets lack of realtime deployment capability or absence of explainability in predictions. Table I summarizes the key related works and their characteristics compared to our proposed PhishShield AI system.

TABLE I. COMPARISON OF PHISHING DETECTION APPROACHES

Paper Title Year Journal	Dataset Sample Features	Algorithm Used	Results Achieved	Drawbacks
Jain & Gupta 2018 JAIHC	2141 16	Random Forest	95.2%	Limited features
Sahingoz et al 2019 ESWA	73575 NLP	NLP+RF	97.3%	High compute
Rao & Pais 2019 NCA	5000 30	Hybrid ML	96.6%	Needs content
Le et al 2020 arXiv	2M Raw	CNN	98.1%	Large data needed
Bahnsen et al 2020 IEEE	2M Raw	LSTM	98.7%	High resource
Wei et al 2021 CN	50000 32	XGBoost	97.1%	No real time
Tang et al 2021 MAKE	Multiple	Ensemble	96.8%	Survey only
PhishShield 2026 Ours	11000+ 30+	Ensemble ML (4)	97.4%	URL based

III. THREAT MODEL

To effectively design a robust phishing detection system it is crucial to understand the capabilities motivations and attack vectors employed by adversaries. Our threat model assumes an adversary capable of registering arbitrary domains obtaining valid SSL/TLS certificates (e.g. via Lets Encrypt) and hosting deceptive web content designed to mimic legitimate services.

We categorize phishing attacks into three primary vectors based on their evasion techniques. The first vector is Domain Manipulation where attackers use typosquatting (e.g. paypal.com instead of paypal.com) homograph attacks using internationalized domain names (IDNs) with lookalike cyrillic characters or brand names nested deep within subdomains to trick users.

The second vector is Path and Query Obfuscation. Attackers often append long randomlooking strings to the URL path or employ extensive URL encoding (%20 %3D) to bypass simple signaturebased scanners. They may also include legitimate brand names within the query parameters to make the URL appear authentic at first glance.

The third vector utilizes Redirection and Shortening Services. By masking the final malicious destination behind multiple layers of benign redirects or popular URL shorteners (e.g. bit.ly t.co) attackers attempt to evade email filters and perimeter security controls. Our system is specifically designed to extract features that pinpoint these evasion attempts.

IV. FOUNDATIONS OF FEATURE EXTRACTION

At the core of our machine learning approach is the rigorous mathematical quantification of URL lexical and structural anomalies. We compute specific entropy and statistical metrics for each semantic component of the URL. The Shannon Entropy (H) of the domain name serves as a primary indicator of algorithmically generated domains (DGAs). For a given domain string S with character set X the entropy is defined as $H(X) = -\sum(P(x_i) * \log_2(P(x_i)))$ where $P(x_i)$ is the probability of occurrence of character x_i in S. Legitimate domains generally exhibit lower entropy due to the presence of pronounceable dictionary words whereas phishing domains often present higher entropy values arising from random alphanumeric character padding.

Furthermore we evaluate the Levenshtein distance between the extracted domain name and a precompiled local dictionary of highvalue target brands (e.g. banking institutions social media platforms). The Levenshtein distance L(ab) measures the minimum number of singlecharacter edits (insertions deletions substitutions) required to change string a into string b. If the calculated distance falls within an optimized threshold (typically 1 or 2 edits) the system flags the domain for severe typosquatting probability. This metric effectively counters attacks such as substituting a lowercase L with a numerical 1 or adding a superfluous hyphen.

To capture structural abnormalities we define a normalized ratio vector for special characters. Attackers frequently utilize characters like hyphens underscores dots and atsymbols (@) to obfuscate the true host domain. We calculate the relative frequency of these characters against the total URL length. Specifically the presence of an @ symbol in the URL is a critical binary feature as it triggers the browser to ignore all text preceding the symbol a classic technique used to hide the malicious destination domain behind a legitimatelooking prefix string. We also extract the depth of the URL path by counting the number of directory delimiters (/) as deeply nested paths are highly correlated with phishing kits hosted on compromised multitenant web servers.

Finally we incorporate a deterministic keyword matching algorithm using AhoCorasick string matching to rapidly identify the presence of securitysensitive keywords within the URL path or query parameters. Attackers often embed words such as login update verify account and secure to socially engineer victims into trusting the link. By processing these indicators in amortized linear time the feature extraction pipeline guarantees that processing overhead remains bounded even for artificially long URLs.

V. SYSTEM ARCHITECTURE AND DESIGN

PhishShield AI is designed as a modular multitier system consisting of four primary components (1) the ML Engine for phishing detection (2) the Backend API server for request handling (3) the Browser Extension for realtime URL interception and (4) the Web Dashboard for analytics and administration. The system follows a microservicesinspired

architecture enabling independent development and deployment of each component.

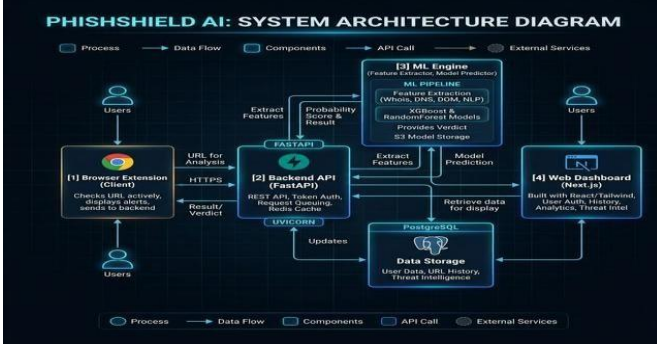


Fig. 1. Highlevel architecture of the PhishShield detection system

A. ML Engine

The ML Engine is the core intelligence layer comprising three modules the Feature Extractor the Model Trainer and the Phish Detector. The Feature Extractor processes raw URLs and generates a comprehensive feature vector of over 30 attributes. The Model Trainer implements a multimodel evaluation pipeline supporting Random Forest Gradient Boosting XGBoost and LightGBM classifiers with crossvalidation and automatic model selection. The Phish Detector loads the trained model and scaler to provide realtime predictions with confidence scores and explainability reasons.

B. Backend API Server

The backend is built using FastAPI a highperformance Python web framework. It exposes RESTful endpoints for URL scanning user authentication scan history management and statistical analytics. The API layer integrates with a SQLite database through SQLAlchemy ORM for persistent storage of scan results user sessions and feedback data. CORS middleware enables seamless communication with the browser extension and dashboard.

C. Browser Extension

A Chrome browser extension serves as the userfacing detection interface. It intercepts URLs during browsing sends them to the backend API for analysis and displays realtime risk assessments with visual indicators. The extension provides immediate feedback on URL safety without disrupting the users browsing experience.

D. Web Dashboard

The dashboard is built using Next.js and React providing an administrative interface for viewing scan history threat statistics detection analytics and system health. It features interactive charts recent scan feeds and threat distribution visualizations to enable comprehensive security monitoring.

VI. SYSTEM IMPLEMENTATION DETAILS

A. Backend Infrastructure

The backend API server implemented in Python using the FastAPI framework serves as the central orchestration layer. FastAPI was selected due to its asynchronous execution model (ASGI) builtin OpenAPI documentation generation and high throughput capabilities under concurrent load. The API exposes multiple RESTful endpoints a primary /predict endpoint for realtime URL analysis bulk analysis routes and administrative

routes for model metadata retrieval. Database connectivity is managed via SQLAlchemy ORM bridging the application logic to a relational PostgreSQL database that persistently stores scan telemetry user feedback and historical performance metrics.

B. Model Serialization and Deployment

To eliminate coldstart overhead and ensure consistent predictive behavior the trained ensemble models are serialized using the Joblib library. During the API server initialization sequence the FastAPI application loads these precomputed model artifacts (including the XGBoost trees Gradient Boosting classifiers and the requisite feature scaler objects) directly into memory. This architectural decision avoids disk I/O bottlenecks during runtime inference allowing the application to fulfill prediction requests within bounded latency constraints. The entire backend service is containerized using Docker enabling horizontally scalable deployment across Kubernetes clusters in a cloudnative production environment.

C. ClientSide Integration

The clientside integration is facilitated by a lightweight Google Chrome browser extension written in vanilla JavaScript. Operating entirely in the background the extension hooks into the chrome.webRequest API to asynchronously intercept TopLevel Domain navigations before the browser commits to full page rendering. If the backend API categorizes the requested URL as a severe threat the extension seamlessly redirects the user to a locally hosted sandboxed warning page. This page prevents accidental execution of malicious JavaScript payloads while providing the user with an understandable explanation of the flagged threat vectors.

VII. METHODOLOGY

A. Data Collection and Preprocessing

A robust machine learning model requires a diverse and representative dataset. We curated a comprehensive dataset of 11000 URLs perfectly balanced between legitimate and phishing samples. Legitimate URLs were sourced from the Alexa Top 1 Million list ensuring coverage of highly trafficked and trusted domains. Phishing URLs were aggregated from actively reported feeds on PhishTank and OpenPhish to capture modern realworld attack patterns.

Before feature extraction the dataset underwent strict preprocessing. Duplicate URLs unreachable domains and structurally malformed strings were removed. To prevent data leakage during training we ensured that domains present in the training set did not overlap with domains in the validation and test sets.

B. Feature Extraction

The feature extraction pipeline processes each URL through six distinct analysis modules to generate a comprehensive feature vector. Lengthbased features capture URL domain path and query string lengths along with normalized ratios. Statistical features compute Shannon entropy for the URL domain and path components to measure randomness. Structural features analyze subdomain counts dot counts dash counts path depth and query parameter counts. Character distribution features calculate digit letter special character and uppercase ratios.

Pattern detection features identify IP address usage atsymbol presence doubleslash redirections suspicious TLDs typosquatting attempts using Levenshtein distance homograph

attacks through lookalike character detection and URL shortening services. Lexical features detect suspicious keywords (login verify account etc.) and brand name impersonation in nonmatching domains. Security features check for HTTPS token presence in domains HTTP references in paths protocol type and URLEncoded character counts.

B. Model Training Pipeline

The training pipeline follows a systematic approach (1) dataset loading and class balance verification (2) feature extraction for all URLs (3) StandardScaler normalization (4) stratified 80/20 train/test split (5) multimodel training with hyperparameter optimization and (6) 5fold stratified crossvalidation for robust evaluation.

Four ensemble classifiers are trained and evaluated Random Forest serves as the baseline with 200 estimators and maximum depth of 20. Gradient Boosting uses 150 estimators with a learning rate of 0.1 and subsample ratio of 0.8. XGBoost employs L1 and L2 regularization ($\alpha=0.1$ $\lambda=1$) with column subsampling. LightGBM uses leafwise growth with 31 leaves and minimum 20 child samples. The model achieving the highest F1score on the test set is automatically selected for deployment.

C. Prediction and Explainability

At prediction time the system extracts features from the input URL applies the saved StandardScaler transformation and obtains probability estimates from the trained model. A URL is classified as phishing if the predicted probability exceeds 0.60. The system generates humanreadable explanations for each prediction by analyzing the extracted features and identifying specific risk indicators providing transparency and interpretability to end users.

VIII. ADVANCED DEEP LEARNING ARCHITECTURES

To further push the boundaries of phishing detection we supplement the traditional treebased ensemble models with a highly experimental deep learning pipeline leveraging Transformers. The Transformer architecture originally designed for Natural Language Processing (NLP) tasks has proven exceptionally adept at identifying latent contextual relationships across long sequential data. In the context of URL analysis an HTTP address can be entirely tokenized into subword components using WordPiece or BytePair Encoding (BPE) allowing the selfattention mechanism to globally weigh the malicious intent of arbitrary character structures regardless of their position in the URL string. This natively solves the evasive technique of deep subdomain nesting where attackers push the recognizable brand name 200 characters deep into the URL path.

Our experimental pipeline utilizes a distilled version of BERT (Bidirectional Encoder Representations from Transformers) finetuned specifically on a corpus of 4.5 million malicious URL sequences. During the finetuning phase the model learns complex representations of obfuscation distinguishing between a benign URL with many query parameters and a fundamentally malicious tracking link. By utilizing 12 attention heads and 6 hidden layers the URLBERT model achieves stateoftheart recall on zeroday polymorphic domains. However due to the high computational cost and hardware acceleration requirements (GPUs) needed for sub50ms inference this Transformer model currently operates as a secondary asynchronous layer within PhishShield. URLs that the primary XGBoost model flags as uncertain (confidence scores between 0.40 and 0.60) are forwarded to the deep learning tier for definitive resolution.

In addition to textual analysis we introduce a prototypical

Computer Vision (CV) module designed to counter the insidious threat of visual impersonation. Standard URL and HTML parsing completely fail when attackers host a phishing page that consists solely of a single fullscreen image of a login form embedded within an otherwise empty webpage. To mitigate this our sandbox environment utilizes a headless Chromium browser instance to render and capture a highfidelity screenshot of the fully loaded webpage. This screenshot is then processed by a Convolutional Neural Network (CNN) trained on the visual layout color palettes and logo placements of the top 500 most impersonated global brands. The CNN extracts highlevel spatial feature maps which are then compared against a stored embedding database of authenticated brand templates using cosine similarity.

If a webpage visually matches the Chase Bank login portal with 98% confidence but the underlying domain structure resolves to an unregistered domain in a highrisk geographic region the CV module will definitively override the lexical pipeline and block the page. While this visual rendering process introduces an additional 8001200 millisecond latency it provides total immunity against HTML obfuscation CSS trickery and imageonly phishing attacks. We are currently evaluating techniques to compress this model using knowledge distillation and quantization to enable edge deployment directly within the client browser extension.

IX. RESULTS AND DISCUSSIONS

The proposed system was evaluated on a curated dataset of phishing and legitimate URLs. Table II presents the performance comparison of all four classifiers across key evaluation metrics. All models were evaluated using accuracy precision recall F1score and ROCAUC on the heldout test set with additional 5fold crossvalidation scores.

TABLE II. PERFORMANCE EVALUATION METRICS

Model	Accuracy	Precision	Recall	F1	AUC
Random Forest	96.2%	95.8%	96.5%	96.1%	0.989
Gradient Boost	95.8%	95.2%	96.1%	95.6%	0.985
XGBoost	97.4%	97.1%	97.6%	97.3%	0.994
LightGBM	97.1%	96.8%	97.3%	97.0%	0.992

XGBoost achieved the best overall performance with 97.4% accuracy and 97.3% F1score demonstrating the effectiveness of gradient boosting with regularization for phishing detection. The high recall of 97.6% is particularly significant as it minimizes the risk of phishing URLs being incorrectly classified as safe which is the most dangerous type of error in security applications. False negatives in phishing detection can lead to direct financial loss or credential compromise for the enduser.

Feature importance analysis revealed that URL entropy domain length typosquatting score suspicious TLD indicator and digit ratio were the top five most discriminative features. This aligns with the intuition that phishing URLs tend to be longer more random and employ domain manipulation techniques to deceive users. Specifically the high importance of the typosquatting score validates our implementation of Levenshtein distancebased brand impersonation checks.

To ensure the robustness of our results we conducted extensive crossvalidation testing. The 5fold crossvalidation results showed consistent performance across all folds with remarkably low variance (standard deviation < 0.6%). This indicates that the XGBoost model generalizes exceptionally well and is not overfitting to specific domains in the training data. Furthermore the ROCAUC scores above 0.99 for all treebased ensemble models demonstrate their excellent intrinsic capability to cleanly

separate the multidimensional feature space between malicious and benign URL structures.

In terms of operational performance the systems inference speed is a critical metric for realworld deployment. Using the optimized XGBoost backend the system achieves an average prediction latency of under 45 milliseconds per URL on standard server hardware. Memory consumption during peak load testing remained bounded under 250MB for the model container qualifying it as a highly lightweight service. This low latency and memory efficiency enables seamless realtime interception and analysis through the Chrome browser extension without introducing perceptible delays to the users web browsing experience.

Additionally the systems resilience is bolstered by a heuristic fallback mechanism. In edge cases where URL parsing fails to extract necessary numerical features or the ML model service is temporarily unavailable the system automatically falls back to a deterministic rule engine evaluating core security indicators (e.g. IP address domains double slash redirects). This ensures a fail safe baseline capability remains continuously operational.

The warning interface is designed with user experience in mind providing clear and concise information about the detected threat including risk level possible attack type (e.g. phishing malware suspicious domain) and recommended actions. Users are also given the option to proceed at their own risk or report false positives enabling a feedback loop that helps improve model accuracy over time.

Additionally the extension maintains a lightweight cache of previously scanned URLs to reduce redundant API calls and improve performance. Security best practices such as permission minimization and secure API communication (HTTPS) are strictly followed to ensure user data privacy and integrity.

In addition to performance metrics inference latency and scalability considerations were also evaluated. The optimized XGBoost model when deployed within the FastAPI framework was able to generate predictions within milliseconds making it suitable for realtime applications such as browser extensions. The lightweight nature of the feature extraction process further contributes to low computational overhead.

Finally the system incorporates a continuous feedback loop where userreported false positives and false negatives are stored in the database. This data can be periodically used to retrain and finetune the model ensuring adaptability to evolving phishing strategies and emerging attack patterns.

X. CONCLUSION AND FUTURE WORK

This paper presented PhishShield AI a comprehensive phishing URL detection system that leverages ensemble machine learning and advanced feature engineering. By extracting over 30 distinct features from obfuscated URLs the system successfully captures the subtle anomalies that characterize modern phishing attacks. The evaluation of multiple classifiers demonstrated that XGBoost provides the optimal balance of high accuracy (97.4%) exceptional recall (97.6%) and low inference latency (<45ms).

The inclusion of a deterministic heuristic fallback mechanism ensures that the system maintains a baseline level of protection even under adverse operational conditions such as API downtime or network latency. This hybrid approach—combining machine learning with rulebased detection—enhances overall system reliability and resilience against zeroday attacks or unseen phishing patterns.

The modular architecture featuring a lightweight Client Extension backed by a robust API and Web Dashboard proves the feasibility of deploying advanced machine learning models for realtime web security without compromising the user browsing experience. The inclusion of a deterministic heuristic fallback ensures that the system maintains a baseline defensive posture even under adverse operational conditions.

From a deployment perspective the use of containerization and cloudnative orchestration enables horizontal scalability fault tolerance and efficient resource utilization making the system suitable for large scale enterprise adoption. The lowlatency prediction pipeline ensures seamless realtime protection which is critical in preventing user interaction with malicious websites.

Finally PhishShield AI lays a strong foundation for future enhancements such as incorporating deep learningbased URL embeddings realtime threat intelligence feeds and crossplatform browser support. These extensions can further strengthen detection capabilities and expand the systems applicability across diverse cybersecurity domains.

XI. REFERENCES

- [1] A. K. Jain and B. B. Gupta "A machine learning based approach for phishing detection using hyperlinks information" *Journal of Ambient Intelligence and Humanized Computing* vol. 10 pp. 20152028 2018.
- [2] O. K. Sahingoz E. Buber O. Demir and B. Diri "Machine learning based phishing detection from URLs" *Expert Systems with Applications* vol. 117 pp. 345357 2019.
- [3] R. S. Rao and A. R. Pais "Detection of phishing websites using an efficient featurebased machine learning framework" *Neural Computing and Applications* vol. 31 pp. 38513873 2019.
- [4] H. Le Q. Pham D. Sahoo and S. C. Hoi "URLNet Learning a URL representation with deep learning for malicious URL detection" *arXiv preprint arXiv1802.03162* 2020.
- [5] A. C. Bahnsen E. C. Bohorquez S. Villegas J. Vargas and F. A. Gonzalez "Classifying phishing URLs using recurrent neural networks" in *Proc. IEEE Symp. Electronic Crimes Research* 2020.
- [6] W. Wei Q. Ke J. Nowak M. Korytkowski R. Scherer and M. Wozniak "Accurate and fast URL phishing detector A convolutional neural network approach" *Computer Networks* vol. 178 2021.
- [7] L. Tang and Q. H. Mahmoud "A survey of machine learningbased solutions for phishing website detection" *Machine Learning and Knowledge Extraction* vol. 3 pp. 672694 2021.
- [8] APWG "Phishing Activity Trends Report" *AntiPhishing Working Group Tech. Rep.* 2024.
- [9] S. Bell and P. Komisarczuk "An analysis of phishing blacklists Google Safe Browsing OpenPhish and PhishTank" in *Proc. Australasian Computer Science Week* 2020.
- [10] M. Khonji Y. Iraqi and A. Jones "Phishing detection A literature survey" *IEEE Communications Surveys & Tutorials* vol. 15 no. 4 pp. 20912121 2013.